实现线程安全的查找表

Posted on 2023-11-26 11:00:02 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2YhAnJl6dnmTK8wboYkwGUQjUqF)

简介

前文介绍了线程安全的队列和栈，本文继续介绍线程安全的查找结构，实现一个类似线程安全的map结构，但是map基于红黑树实现，假设我们要增加或者删除节点，设计思路是依次要删除或增加节点的父节点，然后修改子节点数据 。尽管这种思路可行，但是难度较大，红黑树节点的插入要修改多个节点的关系。另外加锁的流程也是锁父节点，再锁子节点，尽管在处理子节点时我们已经处理完父节点，可以对父节点解锁，继续对子节点加锁，这种情况锁的粒度也不是很精细，考虑用散列表实现。

散列表

散列表（Hash table，也叫哈希表），是根据键（Key）而直接访问在存储器存储位置的数据结构。 也就是说，它通过计算出一个键值的函数，将所需查询的数据映射到表中一个位置来让人访问，这加快了查找速度。 这个映射函数称做散列函数，存放记录的数组称做散列表。

举个例子:

假如我们一共有 50 人参加学校的数学竞赛，然后我们为每个学生分配一个编号，依次是 1 到 50.

如果我们想要快速知道编号对应学生的信息，我们就可以用一个数组来存放学生的信息，编号为 1 的放到数组下标为 1 的位置，编号为 2 的放到数组下标为 2 的位置，依次类推。

现在如果我们想知道编号为 20 的学生的信息，我们只需要把数组下标为 20 的元素取出来就可以了，时间复杂度为 O(1)，是不是效率非常高呢。

但是这些学生肯定来自不同的年级和班级，为了包含更详细的信息，我们在原来编号前边加上年级和班级的信息，比如 030211 ，03 表示年级，02 表示班级，11 原来的编号，这样我们该怎么存储学生的信息，才能够像原来一样使用下标快速查找学生的信息呢?

思路还是和原来一样，我们通过编号作为下标来储存，但是现在编号多出了年级和班级的信

息怎么办呢，我们只需要截取编号的后两位作为数组下标来储存就可以了。

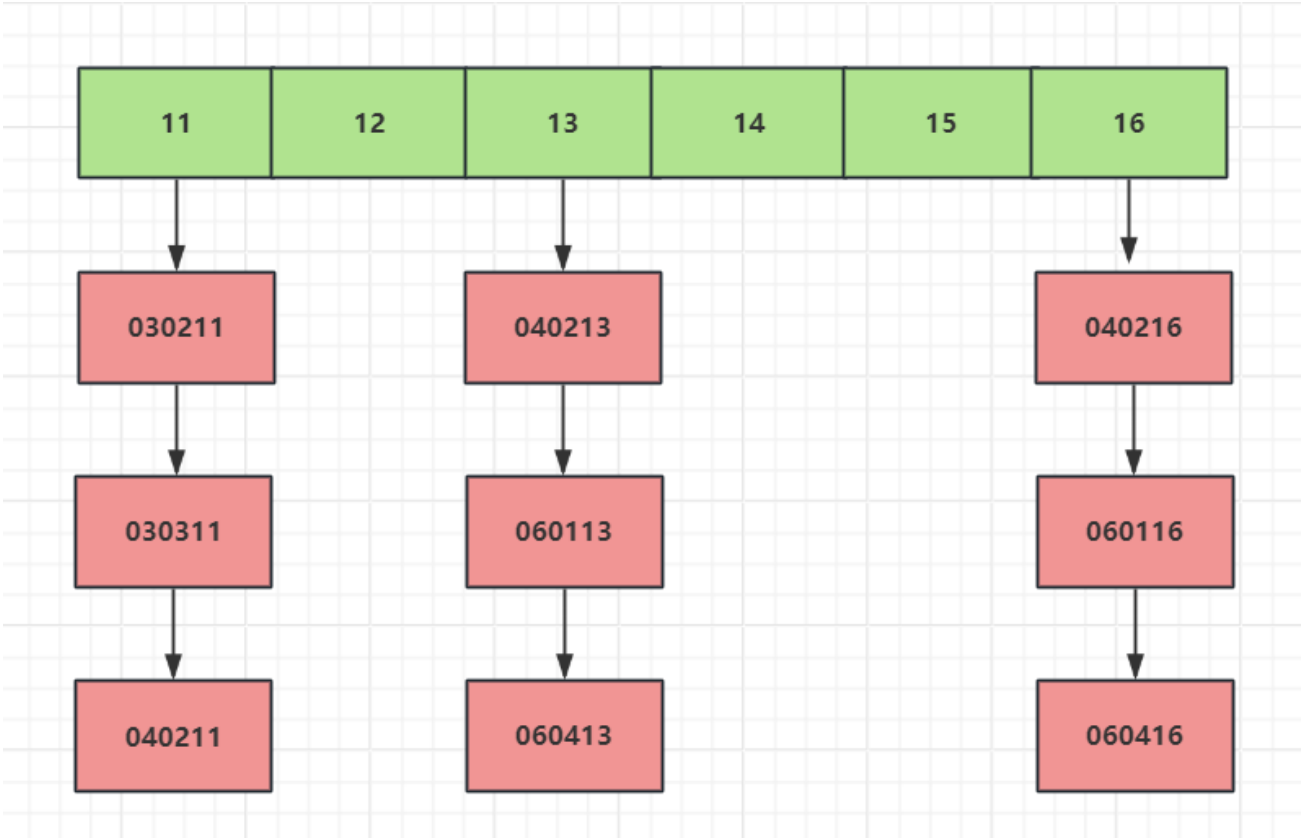
这个过程就是典型的散列思想。其中，参赛学生的编号我们称之为键(key)，我们用它来标识一个学生。然后我们通过一个方法（比如上边的截取编号最后两位数字）把编号转变为数组下标，这个方法叫做散列函数（哈希函数），通过散列函数得到的值叫做散列值（哈希值）。

我们自己在设计散列函数的函数时应该遵循什么规则呢?

1. 得到的散列值是一个非负整数
2. 两个相同的键，通过散列函数计算出的散列值也相同
3. 两个不同的键，计算出的散列值不同

虽然我们在设计的时候要求满足以上三条要求，但对于第三点很难保证所有不同的建都被计算出不同的散列值。有可能不同的建会计算出相同的值，这叫做哈希冲突。最常见的一些解决哈希冲突的方式是开放寻址法和链表法，我们这里根据链表法，将散列函数得到相同的值的key放到同一个链表中。

如下图



当我们根据key值的后两位计算编号，将编号相同的放入一个链表，比如030211和030311是一个编号，所以将其放入一个链表。

同样的道理040213和060113是一个编号，放入一个链表。

设计思路

我们要实现上述逻辑，可以考虑将11，12，13等hash值放入一个vector中。多线程根据key计算得出hash值的过程并不需要加锁，可以实现并行计算。

但是对于链表的增删改查需要加锁。

所以我们考虑将链表封装为一个类 `bucket_type` ,支持数据的增删改查。

我们将整体的查找表封装为 `threadsafe_lookup_table` 类，实现散列规则和调度 `bucket_type` 类。

代码实现

我们先实现内部的 `bucket_type` 类. 为了 `threadsafe_lookup_table` 可以访问他，所以将 `threadsafe_lookup_table` 设置为其友元类。

```
1. class bucket_type
2. {
3.     friend class threadsafe_lookup_table;
4. }
```

我们需要用链表存储键值结构，所以我们可以在 `bucket_type` 中添加一个链表存储键值结构。

```
1.  //存储元素的类型为pair，由key和value构成
2.  typedef std::pair<Key, Value> bucket_value;
3.  //由链表存储元素构
4.  typedef std::list<bucket_value> bucket_data;
5.  //链表的迭代器
6.  typedef typename bucket_data::iterator bucket_iterator;
7.  //链表数据
8.  bucket_data data;
9.  //改用共享锁
10. mutable std::shared_mutex mutex;
```

并且添加了互斥量用于控制链表的读写互斥操作，但是我们采用的是共享互斥量，可以实现

读写锁，保证读的时候可以并发读。

接下来我们封装一个私有的查找接口，用来内部使用。

```
1. //查找key值，找到返回对应的value，未找到则返回默认值
2. bucket_iterator find_entry_for(const Key & key)
3. {
4.     return std::find_if(data.begin(), data.end(),
5.                 [&](bucket_value const& item)
6.                 {return item.first == key; });
7. }
```

然后我们分别实现返回查找的值操作，以及添加操作，并且删除操作

```
1.  //查找key值，找到返回对应的value，未找到则返回默认值
2.  Value value_for(Key const& key, Value const& default_value)
3.  {
4.      std::shared_lock<std::shared_mutex> lock(mutex);
5.      bucket_iterator const found_entry = find_entry_for(key);
6.      return (found_entry == data.end()) ?
7.             default_value : found_entry->second;
8.  }
9.  //添加key和value，找到则更新，没找到则添加
10. void add_or_update_mapping(Key const& key, Value const& value)
11. {
12.     std::unique_lock<std::shared_mutex> lock(mutex);
13.     bucket_iterator const found_entry = find_entry_for(key);
14.     if (found_entry == data.end())
15.     {
16.         data.push_back(bucket_value(key, value));
17.     }
18.     else
19.     {
20.         found_entry->second = value;
21.     }
22. }
23. //删除对应的key
24. void remove_mapping(Key const& key)
25. {
26.     std::unique_lock<std::shared_mutex> lock(mutex);
27.     bucket_iterator const found_entry = find_entry_for(key);
28.     if (found_entry != data.end())
29.     {
30.         data.erase(found_entry);
31.     }
32. }
```

这样我们设计完成了 `bucket_type` 类。

接下来我们设计 `threadsafe_lookup_table` 类。我们用一个vector存储上面的bucket_type类型。 因为我们要计算hash值，key可能是多种类型string, int等，所以我们采用std的hash算法作为散列函数即可.

```
1. class threadsafe_lookup_table{
2. private:
3.     //用vector存储桶类型
4.     std::vector<std::unique_ptr<bucket_type>> buckets;
5.     //hash<Key> 哈希表 用来根据key生成哈希值
6.     Hash hasher;
7.
8.     //根据key生成数字，并对桶的大小取余得到下标，根据下标返回对应的桶智能指针
9.     bucket_type& get_bucket(Key const& key) const
10.    {
11.        std::size_t const bucket_index = hasher(key) % buckets.size();
12.        return *buckets[bucket_index];
13.    }
14. };
```

`get_bucket` 函数不需要加锁，各个线程可以并行计算哈希值，取出key对应的桶。如果多线程调用同一个bucket的增删改查，就通过bucket内部的互斥解决线程安全问题。
接下来我们完善 `threadsafe_lookup_table` 的对外接口

```
threadsafe_lookup_table(
    unsigned num_buckets = 19, Hash const& hasher_ = Hash()) :
    buckets(num_buckets), hasher(hasher_)
{
    for (unsigned i = 0; i < num_buckets; ++i)
    {
        buckets[i].reset(new bucket_type);
    }
}

threadsafe_lookup_table(threadsafe_lookup_table const& other) = delete;
threadsafe_lookup_table& operator=(
    threadsafe_lookup_table const& other) = delete;

Value value_for(Key const& key,
    Value const& default_value = Value())
{
    return get_bucket(key).value_for(key, default_value);
}

void add_or_update_mapping(Key const& key, Value const& value)
{
    get_bucket(key).add_or_update_mapping(key, value);
}

void remove_mapping(Key const& key)
{
    get_bucket(key).remove_mapping(key);
}
```

除此之外我们可将当前查找表的副本作为一个map返回

```
std::map<Key, Value> get_map()
{
    std::vector<std::unique_lock<std::shared_mutex>> locks;
    for (unsigned i = 0; i < buckets.size(); ++i)
    {
        locks.push_back(
                std::unique_lock<std::shared_mutex>(buckets[i]->mutex));
    }
    std::map<Key, Value> res;
    for (unsigned i = 0; i < buckets.size(); ++i)
    {
        //需用typename告诉编译器bucket_type::bucket_iterator是一个类型，以后再实例化
        //当然此处可简写成auto it = buckets[i]->data.begin();
        typename bucket_type::bucket_iterator it = buckets[i]->data.begin();
        for (;it != buckets[i]->data.end();++it)
        {
            res.insert(*it);
        }
    }
    return res;
}
```

测试与分析

我们自定义一个类

```
class MyClass
{
public:
    MyClass(int i):_data(i){}

    friend std::ostream& operator << (std::ostream& os, const MyClass& mc){
        os << mc._data;
        return os;
    }


private:
    int _data;
};
```

接下来我们实现一个函数做测试

```
1. void TestThreadSafeHash() {
2.     std::set<int> removeSet;
3.     threadsafe_lookup_table<int, std::shared_ptr<MyClass>> table;
4.     std::thread t1([&]() {
5.         for(int i = 0; i < 100; i++)
6.         {
7.            auto class_ptr =  std::make_shared<MyClass>(i);
8.             table.add_or_update_mapping(i, class_ptr);
9.         }
10.    });
11.
12.    std::thread t2([&]() {
13.        for (int i = 0; i < 100; )
14.        {
15.            auto find_res = table.value_for(i, nullptr);
16.            if(find_res)
17.            {
18.                table.remove_mapping(i);
19.                removeSet.insert(i);
20.                i++;
21.            }
22.
23.            std::this_thread::sleep_for(std::chrono::milliseconds(10));
24.        }
25.    });
26.
27.    std::thread t3([&]() {
28.        for (int i = 100; i < 200; i++)
29.        {
30.            auto class_ptr = std::make_shared<MyClass>(i);
31.            table.add_or_update_mapping(i, class_ptr);
32.        }
33.        });
34.
35.
36.    t1.join();
37.    t2.join();
38.    t3.join();
39.
40.    for(auto & i : removeSet)
41.    {
42.        std::cout << "remove data is " << i << std::endl;
```

```
    }

   auto copy_map =  table.get_map();
    for(auto & i : copy_map)
    {
        std::cout << "copy data is " << *(i.second) << std::endl;
    }
}
```

t1用来向map中添加数据(从0到99)，t2用来从map中移除数据(从0到99)，如果map中未找到则等待10ms继续尝试，t3则继续继续添加数据(从100到199).
然后分别打印插入的集合和获取的map中的数值。
打印可以看到输出插入集合为(0~99)，copy的map集合为(100~199).

我们分析一下上述查找表的优劣

1 首先我们的查找表可以支持并发读，并发写，并发读的时候不会阻塞其他线程。但是并发写的时候会卡住其他线程。基本的并发读写没有问题。
2 但是对于bucket_type中链表的操作加锁精度并不精细，因为我们采用的是std提供的list容器，所以增删改查等操作都要加同一把锁，导致锁过于粗糙。

下一节会介绍支持并发读写的自定义链表，可以解决bucket_type中的list锁精度不够的短板。

总结

本文介绍了线程安全情况下栈和队列的实现方式

源码链接:

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day15-threadsafehash (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day15-threadsafehash)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(1587) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(577) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

喜欢(587) 浏览(1595)

解密定时器的实现细节

喜欢(566) 浏览(1449)

Linux环境搭建和编码

喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)